02. 핵심 개념

목차

1. 개요
2. Context - 실행 추적 관리자
3. AssignedValue - 할당된 값
4. QuantumCell - 셀 할당 추상화
5. ContextCell - 셀 포인터
6. 관련 문서

개요

halo2-lib의 핵심은 회로 구성을 단순화하고 최적화하는 추상화 계층입니다. 이 문서에서는 회로 작성 시 가장 자주 사용되는 네 가지 핵심 데이터 타입을 다룹니다:

- **Context**: 회로의 실행 추적(execution trace)을 저장하는 가상 컬럼
- **AssignedValue**: 회로 내에서 할당된 셀의 값과 위치를 가리키는 포인터
- **QuantumCell**: advice 컬럼에 값을 할당하는 다양한 시나리오를 추상화한 열거형
- **ContextCell**: 특정 Context 내 셀의 위치를 가리키는 포인터

이 타입들은 모두 `halo2-base/src/lib.rs` 에 정의되어 있으며, halo2-lib의 모든 회로 작성에서 기본이 됩니다.

Context - 실행 추적 관리자

정의

`Context` 는 회로의 단일 실행 스레드를 나타냅니다. 각 Context는 하나의 가상 advice 컬럼을 관리하며, 이 컬럼에 witness 값들과 제약 조건들이 순차적으로 추가됩니다.

```
// 소스: halo2-base/src/lib.rs
pub struct Context<F: ScalarField> {
witness_gen_only: bool,
phase: usize,
type_id: &'static str,
context_id: usize,
pub advice: Vec<Assigned<F>>,
zero_cell: Option<AssignedValue<F>>,
pub selector: Vec<bool>,
pub copy_manager: SharedCopyConstraintManager<F>,
}
```

주요 필드 설명

`witness_gen_only: bool`

증명 생성 시에만 사용되는 플래그입니다:

- `true` : witness 생성만 수행 (증명 생성 단계)
- `false` : proving key와 verification key 생성 (키 생성 단계)

witness 생성 모드에서는 equality constraint나 selector 관리 등 많은 작업을 건너뛸 수 있어 성능이 크게 향상됩니다.

`phase: usize`

Halo2의 challenge phase를 나타냅니다 (일반적으로 0, 1, 2). 각 phase는 verifier로부터 challenge 값을 받을 수 있는 단계를 의미합니다. 대부분의 회로는 phase 0만 사용합니다.

`type_id: &'static str`

이 Context가 속한 가상 영역(virtual region)의 고유 식별자입니다. 서로 다른 가상 영역은 반드시 다른 `type_id` 를 가져야 합니다. 일반적으로 모듈 경로나 크레이트 이름을 접두사로 사용합니다.

`context_id: usize`

같은 `type_id` 를 가진 가상 영역 내에서 이 Context를 식별하는 로컬 카운터입니다. `(type_id, context_id)` 쌍이 전체 회로에서 이 Context를 고유하게 식별합니다.

`advice: Vec<Assigned<F>>`

이 Context의 가상 advice 컬럼입니다. 모든 witness 값과 constant 값이 이 벡터에 순차적으로 추가됩니다. 각 요소는 회로의 한 행(row)에 해당합니다.

`selector: Vec<bool>`

각 행에서 게이트를 활성화할지 여부를 나타내는 selector 컬럼입니다. `witness_gen_only == false` 일 때만 사용됩니다. `selector[i] == true` 이면 해당 행에서 커스텀 게이트가 활성화됩니다.

Context 생성

Context는 `new()` 생성자를 통해 생성됩니다:

```
// 소스: halo2-base/src/lib.rs

impl<F: ScalarField> Context<F> {

pub fn new(

witness_gen_only: bool,

phase: usize,
```

```
type_id: &'static str,
context_id: usize,
copy_manager: SharedCopyConstraintManager<F>,
) -> Self {
Self {
witness_gen_only,
phase,
type_id,
context_id,
advice: Vec::new(),
selector: Vec::new(),
zero_cell: None,
copy_manager,
}
}
}
```

실제로는 `BaseCircuitBuilder` 를 통해 Context를 얻는 것이 일반적입니다:

```
// 예제: Context 얻기
let mut builder = BaseCircuitBuilder::new(false).use_k(10);
let ctx = builder.main(0); // phase 0의 main context 얻기
```

주요 메서드

`assign_cell()` - 셀 할당

`assign_cell()` 은 Context의 advice 컬럼에 새로운 셀을 할당하는 핵심 메서드입니다:

```
// 소스: halo2-base/src/lib.rs
pub fn assign_cell(&mut self, input: impl Into<QuantumCell<F>>) {
match input.into() {
QuantumCell::Existing(acell) => {
self.advice.push(acell.value);
if !self.witness_gen_only {
let new_cell = self.latest_cell();
self.copy_manager
.lock()
.unwrap()
.advice_equalities
.push((new_cell, acell.cell.unwrap()));
}
}
QuantumCell::Witness(val) => {
self.advice.push(Assigned::Trivial(val));
}
QuantumCell::WitnessFraction(val) => {
self.advice.push(val);
```

```
}
QuantumCell::Constant(c) => {
self.advice.push(Assigned::Trivial(c));
if !self.witness_gen_only {
let  new_cell = self.latest_cell();
self.copy_manager.lock().unwrap().constant_equalities.push((c, new_cell));
}
}
}
}
```

이 메서드는 `QuantumCell` 의 변형에 따라 다르게 동작합니다:

- **Existing**: 기존 셀의 값을 복사하고 equality constraint 추가
- **Witness**: 새로운 witness 값을 추가
- **WitnessFraction**: 배치 역원 최적화를 위한 분수 형태로 추가
- **Constant**: 상수 값을 추가하고 fixed 컬럼과의 equality constraint 추가

`load_witness()` - Witness 값 로드

새로운 witness 값을 할당하고 `AssignedValue` 를 반환합니다:

```
// 소스: halo2-base/src/lib.rs
pub  fn  load_witness(&mut  self, witness: F) -> AssignedValue<F> {
self.assign_cell(QuantumCell::Witness(witness));
if !self.witness_gen_only {
```

```
self.selector.resize(self.advice.len(), false);
}
self.last().unwrap()
}
```

사용 예시:

```
// 예제: halo2-base/examples/inner_product.rs 기반
use halo2_base::{Context, gates::flex_gate::GateChip};
use halo2_base::halo2_proofs::halo2curves::bn256::Fr;

fn example(ctx: &mut Context<Fr>, gate: &GateChip<Fr>) {
// Witness 값 로드
let a = ctx.load_witness(Fr::from(5));
let b = ctx.load_witness(Fr::from(3));
// 연산 수행
let c = gate.mul(ctx, a, b); // c = a * b = 15
}
```

`load_constant()` - 상수 값 로드

상수 값을 할당하고 `AssignedValue` 를 반환합니다. 상수는 fixed 컬럼에도 추가되며, advice 셀과 fixed 셀 간에 equality constraint가 생성됩니다:

```
// 소스: halo2-base/src/lib.rs
pub fn load_constant(&mut self, c: F) -> AssignedValue<F> {
```

```
self.assign_cell(QuantumCell::Constant(c));
if !self.witness_gen_only {
self.selector.resize(self.advice.len(), false);
}
self.last().unwrap()
}
```

사용 예시:

```
fn example_constant(ctx: &mut Context<Fr>, gate: &GateChip<Fr>) {
let a = ctx.load_witness(Fr::from(10));
let two = ctx.load_constant(Fr::from(2)); // 상수 2
let result = gate.mul(ctx, a, two); // result = 10 * 2 = 20
}
```

기타 유용한 메서드

`assign_witnesses()` - 여러 witness 값을 한 번에 할당:

```
fn example_batch(ctx: &mut Context<Fr>) {
let values = vec![Fr::from(1), Fr::from(2), Fr::from(3), Fr::from(4)];
let assigned = ctx.assign_witnesses(values);
// assigned는 Vec<AssignedValue<Fr>>
}
```

`get()` - 특정 offset의 셀 가져오기:

```
fn example_get(ctx: &mut Context<Fr>) {
ctx.load_witness(Fr::from(5));
ctx.load_witness(Fr::from(10));
let first = ctx.get(0); // 첫 번째 셀
let last = ctx.get(-1); // 마지막 셀 (음수 인덱싱)
}
```

`constrain_equal()` - 두 셀 간 equality constraint 생성:

```
fn example_equality(ctx: &mut Context<Fr>, gate: &GateChip<Fr>) {
let a = ctx.load_witness(Fr::from(5));
let b = ctx.load_witness(Fr::from(5));
// a와 b가 같다는 제약 조건 추가
ctx.constrain_equal(&a, &b);
}
```

AssignedValue - 할당된 값

정의

`AssignedValue` 는 Context의 advice 컬럼에 할당된 셀을 나타내는 구조체입니다. 값 자체와 그 위치 정보를 모두 포함합니다:

```
// 소스: halo2-base/src/lib.rs
```

```rust
#[derive(Clone, Copy, Debug)]
pub struct AssignedValue<F: crate::ff::Field> {
/// 셀의 값
pub value: Assigned<F>,
/// 셀의 위치 (ContextCell 포인터)
pub cell: Option<ContextCell>,
}
```

필드 설명

`value: Assigned<F>`

셀에 할당된 실제 값입니다. `Assigned<F>` 는 Halo2의 타입으로, 다음 중 하나일 수 있습니다:

- `Assigned::Trivial(F)` : 일반적인 필드 원소 값
- `Assigned::Rational(F, F)` : 분수 형태 (분자, 분모) - 배치 역원 최적화용
- `Assigned::Zero` : 영(zero) 값

`cell: Option<ContextCell>`

셀의 위치를 가리키는 포인터입니다:

- `Some(ContextCell)` : 키 생성 단계에서 셀의 정확한 위치 정보 포함
- `None` : witness 생성 단계에서는 위치 정보가 필요 없으므로 `None`

주요 메서드

`value()` - 값 가져오기

`AssignedValue` 에서 실제 필드 원소 값을 가져옵니다:

```rust
// 소스: halo2-base/src/lib.rs
impl<F: ScalarField> AssignedValue<F> {
pub fn value(&self) -> &F {
match &self.value {
Assigned::Trivial(a) => a,
_ => unreachable!(),
}
}
}
```

사용 예시:

```rust
fn example_value(ctx: &mut Context<Fr>, gate: &GateChip<Fr>) {
let a = ctx.load_witness(Fr::from(42));
// AssignedValue에서 실제 값 가져오기
let val: &Fr = a.value();
println!("값: {:?}", val); // 출력: 값: 42
// 연산에 사용
let b = ctx.load_witness(*val + Fr::from(8));
}
```

가상 셀의 개념

`AssignedValue` 는 "가상 셀"의 개념을 구현합니다. 회로 작성 단계에서는 물리적인 Halo2 테이블의 정확한 위치를 알 필요가 없습니다. 대신:

1. **가상 단계**: Context의 `advice` 벡터에 값을 순차적으로 추가
2. **매핑 단계**: 나중에 `VirtualRegionManager` 가 가상 컬럼들을 물리적 컬럼으로 매핑
3. **참조 유지**: `ContextCell` 포인터를 통해 셀 간 관계(equality constraint 등) 유지

이 추상화 덕분에 개발자는 복잡한 컬럼 관리 없이 회로 로직에만 집중할 수 있습니다.

QuantumCell - 셀 할당 추상화

정의

`QuantumCell` 은 advice 컬럼에 값을 할당하는 다양한 시나리오를 추상화한 열거형입니다:

```rust
// 소스: halo2-base/src/lib.rs
#[derive(Clone, Copy, Debug)]
pub enum QuantumCell<F: ScalarField> {
/// 이미 존재하는 AssignedValue 재사용
Existing(AssignedValue<F>),
/// 새로운 witness 값
Witness(F),
/// 배치 역원을 위한 분수 형태의 witness
WitnessFraction(Assigned<F>),
/// 상수 값 (fixed 컬럼에도 추가됨)
Constant(F),
}
```

변형 상세 설명

`Existing` - 기존 셀 재사용

이미 할당된 `AssignedValue` 를 재사용합니다. 새로운 셀이 생성되고, 기존 셀과 새 셀 간에 equality constraint가 자동으로 추가됩니다.

사용 시나리오:

- 같은 값을 여러 곳에서 사용해야 할 때
- 한 Context의 출력을 다른 연산의 입력으로 사용할 때

코드 예시:

```
use halo2_base::QuantumCell::Existing;


fn example_existing(ctx: &mut Context<Fr>, gate: &GateChip<Fr>) {
let a = ctx.load_witness(Fr::from(5));
let b = ctx.load_witness(Fr::from(3));
// a를 여러 번 사용
let c = gate.mul(ctx, a, b); // c = a * b
let d = gate.add(ctx, a, c); // d = a + c
// inner_product에서 Existing 사용
let vec_a = vec![a, b, c];
let vec_b = vec![ctx.load_witness(Fr::from(1)),
ctx.load_witness(Fr::from(2)),
ctx.load_witness(Fr::from(3))];
// vec_b의 값들을 Existing으로 변환하여 재사용
```

```
let result = gate.inner_product(
ctx,
vec_a,
vec_b.into_iter().map(Existing)
);
}
```

`Witness` - 새로운 Witness 값

증명자만 아는 비밀 입력값을 나타냅니다. 가장 일반적으로 사용되는 변형입니다.

사용 시나리오:

- 사용자의 비밀 입력 (개인키, 비밀번호 등)
- 중간 계산 결과
- 회로의 주요 입력값

코드 예시:

```
use halo2_base::QuantumCell::Witness;

fn example_witness(ctx: &mut Context<Fr>) {
// 직접 Witness 사용
ctx.assign_cell(Witness(Fr::from(42)));
// load_witness()는 내부적으로 Witness를 사용
let a = ctx.load_witness(Fr::from(100));
// 여러 witness 값 할당
```

```
let values = vec![Fr::from(1), Fr::from(2), Fr::from(3)];
let assigned = ctx.assign_witnesses(values);
}
```

`WitnessFraction` - 배치 역원용 분수

나눗셈 연산을 최적화하기 위한 특수한 형태입니다. 여러 나눗셈을 한 번에 처리하는 "배치 역원 (batch inversion)" 기법에 사용됩니다.

사용 시나리오:

- 타원곡선 점 연산에서의 나눗셈
- 여러 역원 계산이 필요한 경우
- 내부 최적화 (일반 사용자는 직접 사용할 일이 거의 없음)

코드 예시:

```
use halo2_proofs::plonk::Assigned;

fn example_fraction(ctx: &mut Context<Fr>) {
// 분수 형태로 값 할당 (분자 / 분모)
let numerator = Fr::from(10);
let denominator = Fr::from(3);
ctx.assign_cell(QuantumCell::WitnessFraction(
Assigned::Rational(numerator, denominator)
));
// 나중에 배치 역원 처리 시 실제 값으로 평가됨
```

```
}
```

주의: `WitnessFraction` 에 대해 `value()` 를 호출하면 패닉이 발생합니다. 배치 역원 처리 후에만 값을 가져올 수 있습니다.

`Constant` - 상수 값

회로 생성 시점에 알려진 공개 상수입니다. Verifier도 이 값을 알 수 있습니다.

사용 시나리오:

- 알고리즘의 고정 상수 (예: 암호학적 상수)
- 공개 파라미터
- 0, 1 같은 자주 사용되는 값

코드 예시:

```rust
use halo2_base::QuantumCell::Constant;


fn example_constant(ctx: &mut Context<Fr>, gate: &GateChip<Fr>) {
// 직접 Constant 사용
ctx.assign_cell(Constant(Fr::from(2)));
// load_constant()는 내부적으로 Constant를 사용
let two = ctx.load_constant(Fr::from(2));
let pi_approx = ctx.load_constant(Fr::from(314159)); // 3.14159 * 100000
// 상수를 연산에 사용
let a = ctx.load_witness(Fr::from(10));
let result = gate.mul(ctx, a, two); // result = 10 * 2
```

```
}
```

특징:

- Fixed 컬럼에도 같은 값이 추가됨
- Advice 셀과 Fixed 셀 간에 equality constraint 자동 생성
- Verifier가 값을 검증할 수 있음

QuantumCell 변환

`AssignedValue` 는 자동으로 `QuantumCell::Existing` 으로 변환됩니다:

```
// 소스: halo2-base/src/lib.rs
impl<F: ScalarField> From<AssignedValue<F>> for QuantumCell<F> {
fn from(a: AssignedValue<F>) -> Self {
Self::Existing(a)
}
}
```

이를 통해 `AssignedValue` 를 `QuantumCell` 을 받는 함수에 직접 전달할 수 있습니다:

```
fn example_conversion(ctx: &mut Context<Fr>, gate: &GateChip<Fr>) {
let a = ctx.load_witness(Fr::from(5));
// a는 자동으로 QuantumCell::Existing(a)로 변환됨
ctx.assign_cell(a);
}
```

ContextCell - 셀 포인터

정의

`ContextCell` 은 특정 Context 내 셀의 위치를 가리키는 포인터 구조체입니다:

```rust
// 소스: halo2-base/src/lib.rs
#[derive(Clone, Copy, Debug, Hash, PartialEq, Eq, PartialOrd, Ord)]
pub struct ContextCell {
/// 이 셀이 속한 가상 영역의 고유 식별자
pub type_id: &'static str,
/// 이 셀이 속한 Context의 식별자
pub context_id: usize,
/// Context의 advice 컬럼 내에서 이 셀의 상대적 위치
pub offset: usize,
}
```

필드 설명

`type_id: &'static str`

셀이 속한 가상 영역의 고유 식별자입니다. 서로 다른 가상 영역(예: FlexGate, RangeGate)은 서로 다른 `type_id` 를 가집니다.

`context_id: usize`

같은 가상 영역 내에서 Context를 구분하는 식별자입니다. 예를 들어, 병렬 처리를 위해 여러 Context를 생성할 때 각각 다른 `context_id` 를 가집니다.

`offset: usize`

Context의 `advice` 벡터 내에서 셀의 인덱스입니다. `advice[offset]` 이 이 셀의 값입니다.

포인터로서의 역할

`ContextCell` 은 다음과 같은 상황에서 셀을 참조하는 데 사용됩니다:

1. **Equality Constraints**: 두 셀이 같은 값을 가져야 한다는 제약 조건
2. **Copy Constraints**: 한 셀의 값을 다른 위치로 복사
3. **가상-물리 매핑**: 가상 컬럼의 셀을 물리적 Halo2 테이블의 셀로 매핑

코드 예시:

```
// ContextCell 생성
let  cell = ContextCell::new("my_virtual_region", 0, 5);
// type_id = "my_virtual_region"
// context_id = 0
// offset = 5 (advice 벡터의 6번째 요소)


// AssignedValue는 내부적으로 ContextCell을 포함
let  a = ctx.load_witness(Fr::from(42));
// a.cell = Some(ContextCell { type_id: "...", context_id: 0, offset: 0 })
```

실제 사용 예시

`ContextCell` 은 주로 내부적으로 사용되지만, equality constraint 관리에서 그 역할을 볼 수 있습니다:

```
// Context의 assign_cell() 메서드에서 (소스: halo2-base/src/lib.rs)

QuantumCell::Existing(acell) => {

self.advice.push(acell.value);

if !self.witness_gen_only {

let  new_cell = self.latest_cell(); // 새로 추가된 셀의 ContextCell

self.copy_manager

.lock()

.unwrap()

.advice_equalities

.push((new_cell, acell.cell.unwrap())); // 두 ContextCell 간 equality

}

}
```

이 코드는:

1. 새 셀을 `advice` 벡터에 추가
2. 새 셀의 `ContextCell` 포인터 생성 (`new_cell`)
3. 기존 셀의 `ContextCell` 포인터 가져오기 (`acell.cell.unwrap()`)
4. 두 포인터를 `advice_equalities` 에 추가하여 equality constraint 등록

ContextTag

`ContextCell` 과 관련된 타입으로 `ContextTag` 가 있습니다:

```
// 소스: halo2-base/src/lib.rs
```

```
pub type ContextTag = (&'static str, usize);
```

`ContextTag` 는 `(type_id, context_id)` 쌍으로, 전체 회로에서 Context를 고유하게 식별합니다:

```
impl<F: ScalarField> Context<F> {
pub fn tag(&self) -> ContextTag {
(self.type_id, self.context_id)
}
}
```

관련 문서

- 01. 개요 및 아키텍처 - halo2-lib의 전체 구조
- 03. 회로 구성 프로세스 - BaseCircuitBuilder와 Context 사용법
- 04. 제약 조건 시스템 - FlexGate와 RangeGate에서 이 타입들의 활용
- 05. 가상 영역 관리 - ContextCell의 물리적 매핑 과정

다음 단계

핵심 타입들을 이해했다면, 이제 실제로 회로를 구성하는 방법을 학습하세요:

→ **03. 회로 구성 프로세스**: BaseCircuitBuilder를 사용한 회로 생성 전체 과정

참고 자료

- **소스 파일**: `halo2-base/src/lib.rs`
- **예제**: `halo2-base/examples/inner_product.rs`

- **관련 모듈**: `halo2-base/src/virtual_region/`